# How one Diaspore run happens

A simple walkthrough with a key-value store

## The idea

Diaspore runs your programs inside a fake world that it controls: the clock, the network, and the crashes. One number, called the **seed**, decides every random choice. The same seed gives the same run every time, so a bug that appeared once can be replayed.

Gray boxes answer a common doubt. Orange boxes mark decisions the team has not made yet.

## What you bring

- `kvnode`: your key-value program. Three copies run as **nodes** *(floret)*. A node is one program in the system being tested.
- `diaspore.yaml`: a config file that says what to simulate. Here it says: 3 nodes, random delays, random crashes, and one rule to check.

**Doubt: Does Diaspore run my code inside itself?**
No. It starts each node as a separate program and talks to it by JSON lines, one event in and one reply out.

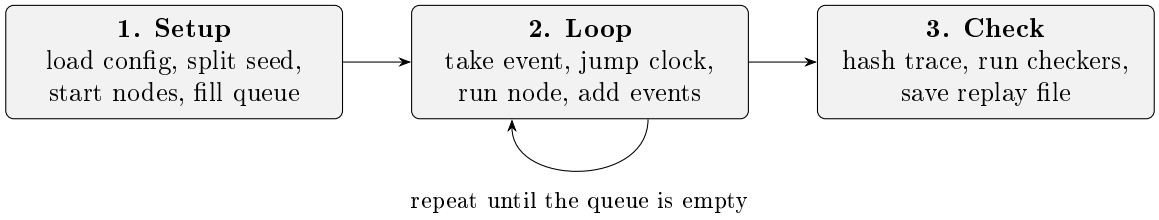
**Doubt: Can real Redis use Diaspore?**
Not in v0.1. Diaspore cannot test an existing service unchanged, so a node must follow the Diaspore protocol.

You start a run with one command:

```
diaspore run --seed 4217
```

**Still open (not set): What if you run without `--seed`?**
Require the flag, or pick one number from the real clock, print it, and save it. The project files do not say.

## The run has three phases




### Phase 1: setup

1. Diaspore reads `diaspore.yaml` and the seed 4217.
2. It splits the seed into private random streams: one for the network, one for the faults, one for the workload, and one for each node.
3. It starts three `kvnode` processes. It sends each one an `init` event, and each answers `done`.

4. It fills the **event queue**, a list of events sorted by time, with what it already knows: the client's put, a crash of `n1`, its restart, and a later get.

**Doubt: Who generates the seed?**
You pick it. It is just a number, like 4217. A seed sweep *(dandelion)* walks through a range of numbers, one run each.

**Doubt: Who writes the event queue?**
Nobody. Diaspore builds it while the run goes. The seed fills in the details, and node replies add new events during the loop.

**Doubt: What is a client?**
A virtual user inside Diaspore. It is not a node. The seeded workload makes it send requests, and Diaspore records what each one got back.

### Phase 2: the loop

Each pass does four things:

1. Take the earliest event from the queue.
2. Jump the virtual clock to that event's time.
3. Send the event to one node and wait for its `done` reply. Diaspore handles crashes and client messages itself.
4. Apply the reply: draw a delay for each message, maybe drop it, add the new events to the queue, and write one line to the trace.

Here is our run. The numbers are invented to make the bug easy to see.

| Pass | Clock | Event taken | What happens |
|---|---|---|---|
| 1 | 0 → 1 | `put x=5` arrives at `n1` | `n1` replies `ok` and sets a 10 ms timer. It has not saved `x` yet. |
| 2 | 1 → 2 | `ok` arrives at the client | Diaspore records that the put got `ok`. |
| 3 | 2 → 6 | crash `n1` | The process is killed. Memory is gone, the timer is lost, nothing was saved. |
| 4 | 6 → 1006 | restart `n1` | New process. `init` arrives with empty saved data, so `x=5` is gone. |
| 5 | 1006 → 1101 | `get x` arrives at `n1` | `n1` has no `x`, so it replies `null`. |
| 6 | 1101 → 1102 | `null` arrives at the client | Diaspore records that the get returned `null`. The queue is empty. |

**Doubt: How can 1000 ms pass in no real time?**
The clock only jumps from event to event, like going from pass 3 to pass 4. Nothing waits.

**Doubt: Why does only one node run at a time?**
It is called lockstep. It keeps two runs of one seed identical, because nothing can race. The cost is speed.

**Doubt: Does Diaspore detect faults, like the crash in pass 3?**
No. Diaspore creates them on purpose, using the seed. The checkers then judge how the system behaved, and the trace tells you which fault caused it.

**Still open (CP11): What does a crash do to timers?**
This doc assumes they are lost, as in pass 3. The team decides at CP11.

**Still open (CP15): Are messages that cross a partition dropped or held?**
This run has no partition. Both choices are possible, and the team decides at CP15.

### Phase 3: check

- Diaspore hashes the trace (one line per event) into one short fingerprint. The same seed and the same node code give the same fingerprint.
- The `acked-writes` checker reads the history: `put x=5` got `ok`, and later `get x` got `null`. The rule "an acknowledged write is never lost" is broken, so the run fails.
- Diaspore saves a **replay file** *(pappus)* with the seed, the config, a hash of the node program, and the expected trace hash.

To replay the failure, type `diaspore replay` with that file. Diaspore reruns the same run from the saved inputs. If the new trace hash matches the saved one, the replay is exact.

**Doubt: If I change one line of node code, do old seeds still work?**
They still run, but not the same run. The replies change, so the queue and the trace change. Rerun the sweep.

**Still open (not set): How is a fixed node checked against an old replay file?**
The project files do not say. The file stores a hash of the node program, so Diaspore can tell the code has changed.

**Still open (not set): Can users add their own checkers?**
v0.1 ships two: `acked-writes` and `linearizability`. Custom checkers are not mentioned.

## Who decides what

| Who | What they decide |
|---|---|
| You | The rules: the node code, and the config (how many nodes, which faults, which checker). |
| The seed | The exact numbers: delays, drops, crash times, what the clients ask. |
| Diaspore | The work: it builds the queue, runs the loop, applies the faults, and records everything. |

The times, the history, and the JSON fields are examples. The protocol fields come from the build guide draft and may change at CP7.